



UniConnect

UniConnect Project Documentation

A complete and practical guide to the internship and talent platform

Prepared by: Gaweeshha Bhagyajith Kumarathunga

Date: 8th October 2026

Repository: [GitHub_Repository_Link](#)

This document describes the implemented UniConnect system and is intended for project reviewers, developers, and future maintainers.

Document Guide

The document is organized into the following sections so readers can quickly find the information they need.

1. Project background, goals, users, and completed features
2. Technology stack, architecture, and security design
3. Use case, class, and entity relationship diagrams
4. Frontend experience and annotated application screenshots
5. REST API, installation, testing, limitations, and future work

Project in one paragraph

UniConnect is a full-stack internship platform that brings students and employers into one clear workflow. Students can build a professional profile, manage education and skills, upload a CV, discover internships, save opportunities, apply, and follow each decision. Companies can publish internships, review complete candidate profiles, download CVs, and update application progress. Administrators can monitor the platform and moderate accounts and listings. The result is a practical system that replaces scattered messages and spreadsheets with one secure, responsive application.

1. Project Overview

Finding an internship often involves several disconnected tools. A student may keep profile information in one place, send a CV through email, and then wait for status updates that are difficult to track. Employers face the same fragmentation when they publish vacancies, collect applications, and compare candidates.

UniConnect was created to make that process feel simpler. It gives students, companies, and administrators a workspace designed around their actual tasks. Instead of treating internship recruitment as a collection of unrelated forms, the system connects profiles, opportunities, applications, status history, notifications, and account controls into one continuous experience.

1.1 Project Goals

- Give students one reusable professional profile for internship applications.
- Help students find relevant opportunities and understand where each application stands.
- Give companies a clear way to publish internships and review suitable candidates.
- Protect personal and business data with role-aware, token-based security.
- Provide an interface that remains comfortable on desktop, tablet, and mobile screens.
- Keep the codebase understandable through separate frontend, controller, service, repository, and persistence layers.

1.2 Intended Users

User	What the user needs	How UniConnect helps
Student	A professional presence, suitable internships, and visible application progress.	Profile and portfolio tools, CV management, search, saving, applications, timelines, and notifications.
Company	A reliable way to publish roles and manage candidates.	Company profile, internship management, candidate review, CV download, and status controls.
Administrator	A broad view of activity and safe moderation tools.	Platform statistics, user activation controls, and internship moderation.

2. Completed Functional Scope

The application covers the core journey from account creation to a company decision. The following features are present in the current implementation.

2.1 Student Experience

- Register, sign in, sign out, and maintain account details securely.
- Create and edit university, degree, graduation year, phone, and biography details.
- Add education records and maintain a reusable list of skills.
- Upload, download, replace, or remove a PDF CV up to 5 MB.
- Search, filter, sort, and browse current internship opportunities.
- Save interesting internships and remove them later.
- Apply once to an open internship and prevent duplicate or late submissions.
- Track application status and review the full decision timeline.
- View dashboard statistics, profile completion, and notifications.

2.2 Company Experience

- Maintain the employer profile, industry, website, description, and contact details.
- Create, edit, review, and delete company-owned internships.
- Review applications only for internships owned by the signed-in company.
- Open a candidate profile with contact details, education, skills, and CV availability.
- Download candidate CVs and move an application through the recruitment stages.
- Review dashboard totals and receive a notification when a student applies.

2.3 Administrator Experience

- Review totals for users, students, companies, internships, and applications.
- Activate or deactivate user accounts when moderation is required.
- Remove inappropriate internship listings.
- Prevent an administrator from deactivating the account currently being used.

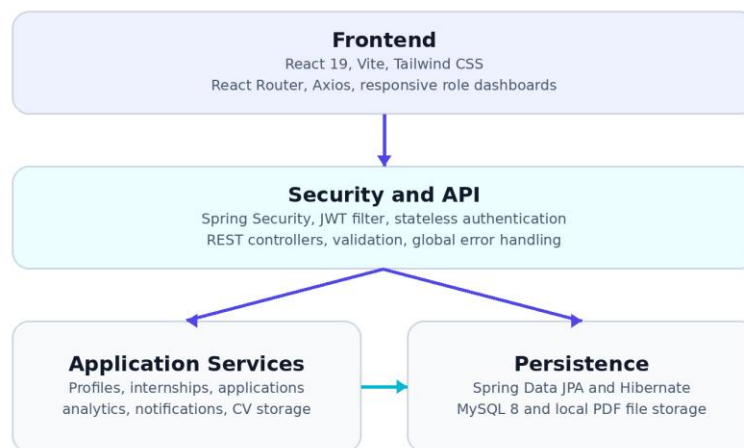
3. Technology and Architecture

Layer	Technology	Role in the project
Frontend	React 19 and Vite 8	Builds the interactive single-page interface and role-specific screens.
Navigation	React Router 7	Protects routes and directs each role to the correct workspace.
Styling	Tailwind CSS 4 and custom CSS	Provides the visual system, responsive layouts, and dark mode.
HTTP	Axios	Calls the backend and automatically attaches the JWT to protected requests.
Backend	Spring Boot 4.1.1 and Java 24	Hosts REST endpoints, validation, business rules, and configuration.
Security	Spring Security, BCrypt, JWT 0.13	Protects credentials and validates stateless authentication tokens.
Persistence	Spring Data JPA and Hibernate	Maps Java entities to relational tables and manages transactions.
Database	MySQL 8	Stores accounts, profiles, internships, applications, and related records.

3.1 High-Level Architecture

The React client sends JSON requests to a Spring Boot REST API. A security filter validates the Bearer token before protected controller methods run. Controllers validate input and pass work to services. Services apply business rules and use repositories to read or write MySQL data. CV documents are saved on the server file system, while the student table keeps the file metadata.

UniConnect System Architecture



Client requests use JSON over HTTP. Protected requests include a Bearer JWT.

Figure 1. High-level UniConnect system architecture

3.2 Typical Protected Request

1. The user signs in with an email address and password.
2. The backend verifies the BCrypt password hash and returns a JWT containing the email, user ID, and role.
3. The frontend stores the token and sends it as an Authorization Bearer header on later requests.
4. The JWT filter validates the signature and expiry time, checks that the account is active, and creates the authentication context.
5. The controller and service confirm the role and ownership before returning or changing protected data.

4. Use Case Model

The use case model shows what each role can do from a user's point of view. Shared actions such as sign-in, notifications, and settings are available across the role-specific workspaces, while recruitment actions remain protected by role and ownership rules.

UniConnect Use Case Diagram

The main actions available to students, companies, and administrators.

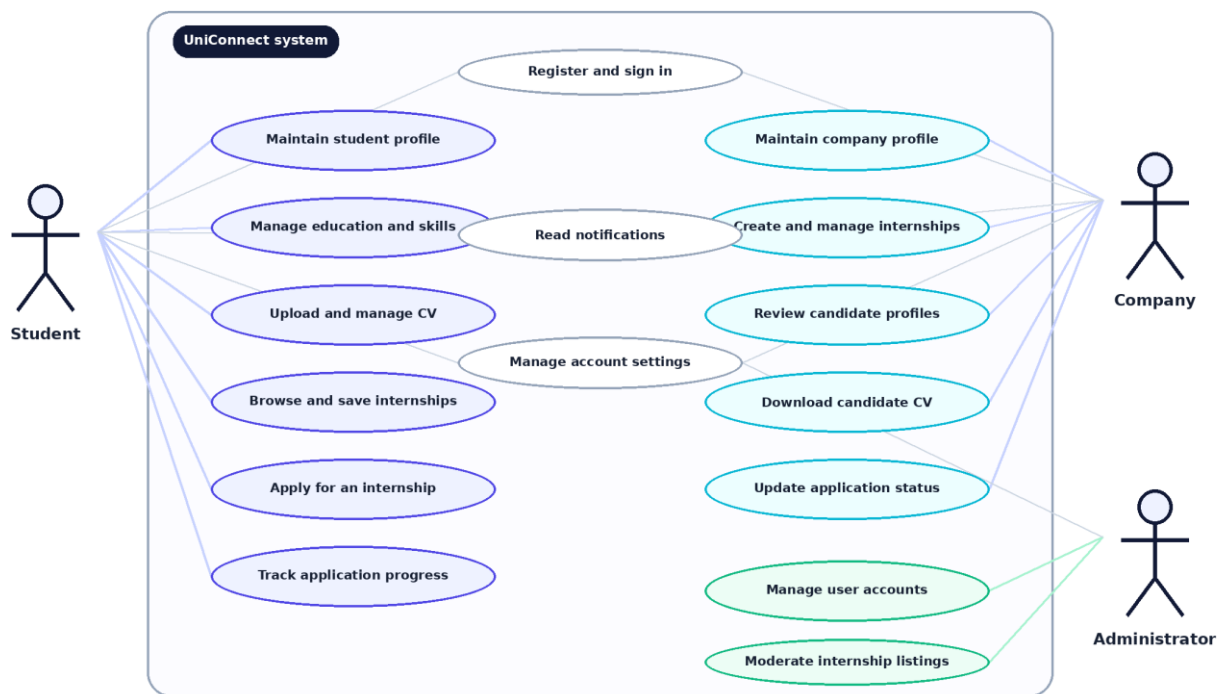


Figure 2. Completed UniConnect use case diagram

Use case	Primary actor	Successful outcome
Apply for internship	Student	A unique pending application is created and the company is notified.
Manage internship	Company	The company publishes or updates a listing it owns.
Review candidate	Company	The employer sees the student's relevant profile, education, skills, and CV state.
Update application status	Company	The new status is saved, history is recorded, and the student is notified.
Moderate account	Administrator	The selected user account becomes active or inactive.
Moderate listing	Administrator	An unsuitable internship is removed from the platform.

5. Class Design

The backend domain model mirrors the language of the internship process. A User owns the login identity and role. Student and Company add role-specific profile information. Internship, InternshipApplication, and ApplicationStatusHistory carry the recruitment workflow, while the remaining classes support portfolios, bookmarks, and notifications.

UniConnect Domain Class Diagram

A simplified view of the main JPA entities and their responsibilities.

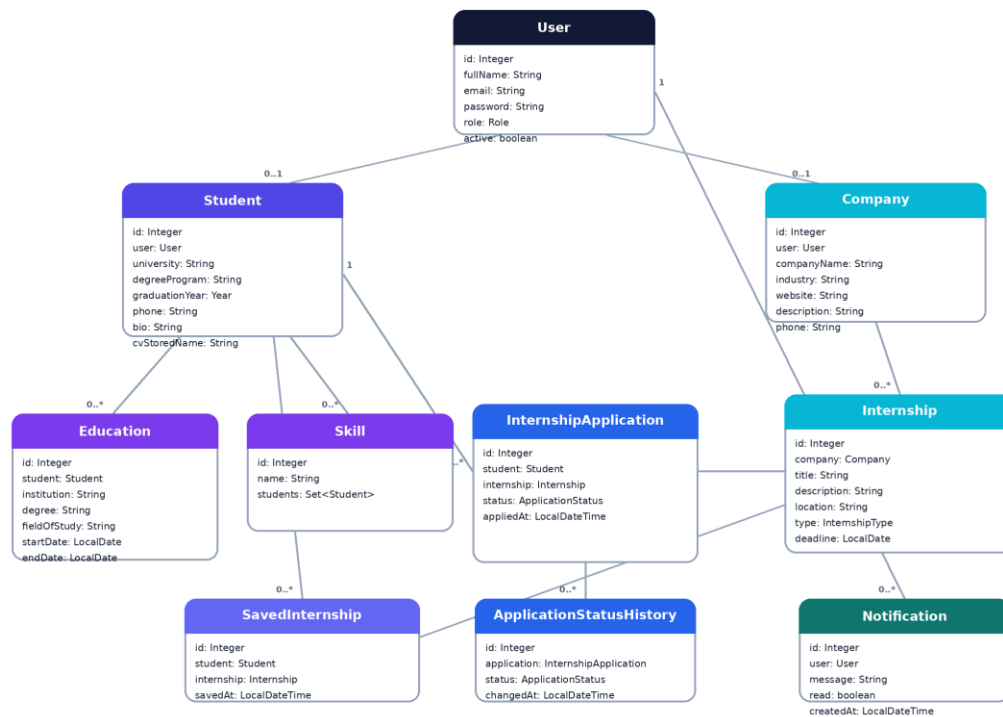


Figure 3. Simplified domain class diagram

Class	Responsibility
User	Stores shared account identity, credentials, role, active state, and creation time.
Student	Stores the student profile and CV metadata and connects to education and skills.
Company	Stores employer profile information and owns internship listings.
Internship	Describes an opportunity, work type, location, deadline, and owner.
InternshipApplication	Connects one student to one internship and stores the current decision.
ApplicationStatusHistory	Keeps the chronological record of application decisions.
SavedInternship	Records a student's bookmarked opportunity.
Notification	Stores user-facing messages and their read state.

6. Database Design

MySQL stores the application's structured data. The schema keeps login identity separate from student and company profiles, uses foreign keys to preserve relationships, and uses unique constraints to prevent repeated email addresses, duplicate applications, and duplicate saved internships.

UniConnect Entity Relationship Diagram

Primary keys, foreign keys, and cardinalities in the implemented relational model.

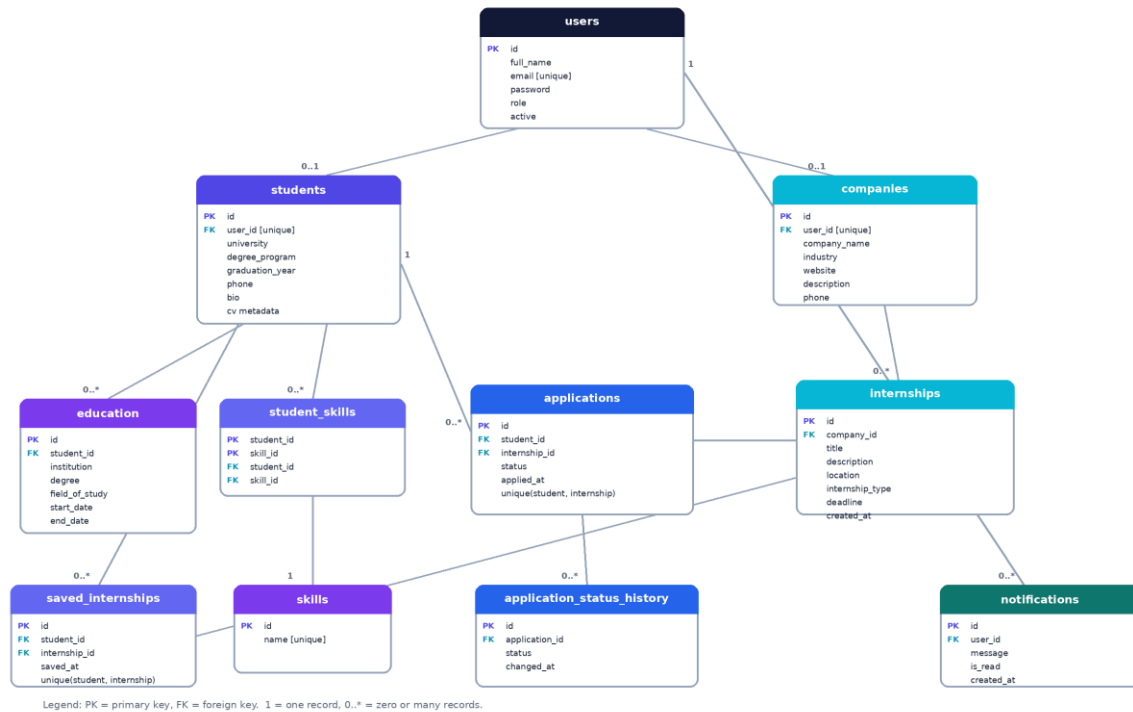


Figure 4. Completed entity relationship diagram

Table	Purpose	Important rule
users	Shared authentication and role information.	Email is unique; password stores a BCrypt hash.
students	Student profile and CV metadata.	One optional profile per user.
companies	Company profile and contact details.	One optional profile per user.
education	Student education history.	Many records may belong to one student.
skills / student_skills	Reusable skills and the student-to-skill relationship.	A skill name is unique.
internships	Company-owned opportunities.	A company may own many listings.
applications	Student submissions and current statuses.	Student and internship pair is unique.
application_status_history	Timeline of every application status change.	Many timeline entries may belong to one application.
saved_internships	Student bookmarks.	Student and internship pair is unique.
notifications	User messages and read state.	Many notifications may belong to one user.

7. Security and Data Protection

Security is handled at several levels. Passwords are never stored as readable text. JWTs are signed using a secret taken from the environment, and protected endpoints require a valid token. The service layer also checks who owns a profile, internship, or application before allowing changes. This matters because authentication alone does not prove that a user should be able to edit a particular record.

- BCrypt hashes account passwords before they are stored.
- JWT authentication keeps the API stateless and includes an expiry time.
- Student, company, and administrator actions are separated by role.
- Ownership checks prevent one company from changing another company's internships or applications.
- Inactive accounts cannot continue using protected features.
- Validation rejects missing fields, malformed values, invalid enums, late applications, and unsupported CV files.
- Database and JWT secrets are read from environment variables and should never be committed to GitHub.

Important security note

A JWT is a temporary credential. It should be treated like a password while it is valid. Screenshots, commits, and issue reports should never include a real token or environment secret.

8. REST API Overview

The frontend communicates with the backend through JSON REST endpoints. The table below highlights the main routes used by the completed interface. Public authentication routes do not require a token; the remaining routes require a Bearer JWT and the correct role.

8.1 Authentication and Profiles

Method	Endpoint	Purpose
POST	/api/auth/register	Create a student or company account.
POST	/api/auth/login	Authenticate and return the user's JWT and role.
GET/POST/PUT	/api/students/profile	Read, create, or update the signed-in student profile.
GET/POST/PUT	/api/companies/profile	Read, create, or update the signed-in company profile.
GET/PUT	/api/users/me	Read or update shared account details.

8.2 Opportunities and Applications

Method	Endpoint	Purpose
GET	/api/internships	Browse all internships.
POST	/api/internships	Create a company-owned internship.
GET	/api/internships/company/my	List the signed-in company's internships.
PUT/DELETE	/api/internships/{id}	Update or delete an owned internship.
POST	/api/applications	Submit one student application.
GET	/api/applications/student/my	List the signed-in student's applications.
GET	/api/applications/company/my	List applications received by the company.
PUT	/api/applications/{id}/status	Change status and record history.

8.3 Portfolio, Engagement, and Administration

Method	Endpoint	Purpose
CRUD	/api/students/education	Manage the student's education entries.
GET/PUT	/api/students/skills	Read or replace the student's skill list.
POST/GET/DELETE	/api/students/cv	Upload, download, or remove the student's PDF CV.
POST/GET/DELETE	/api/saved-internships	Manage the student's saved opportunities.
GET/PUT	/api/notifications	Read notifications and mark them as read.
GET/PUT/DELETE	/api/admin/**	View statistics and moderate users or internships.

9. Interface and User Experience

The visual design uses a dark navy workspace, bright indigo-to-blue accents, clear status colors, and consistent rounded panels. Student and employer screens share the same design language, but their navigation and dashboard content reflect different priorities. A light and dark theme toggle is available, and the chosen preference is kept in the browser.

Design choice	Reason
Role-specific side navigation	Keeps the most useful actions visible without mixing student and employer tasks.
Dashboard summary cards	Lets users understand profile progress and recruitment activity at a glance.
Status badges and timelines	Turns application progress into information that is easy to scan.
Search, filters, and sorting	Helps students narrow opportunities without leaving the internship page.
Responsive grids and mobile drawer	Allows the same workflow to remain usable on smaller screens.
Loading, empty, success, and error states	Explains what is happening instead of leaving a blank or confusing screen.
Accessible form labels and touch targets	Makes forms easier to understand and controls easier to use.

10. Application Screenshots

The following screenshots were captured from the completed frontend. Together they show the main student and employer journeys in the current build.

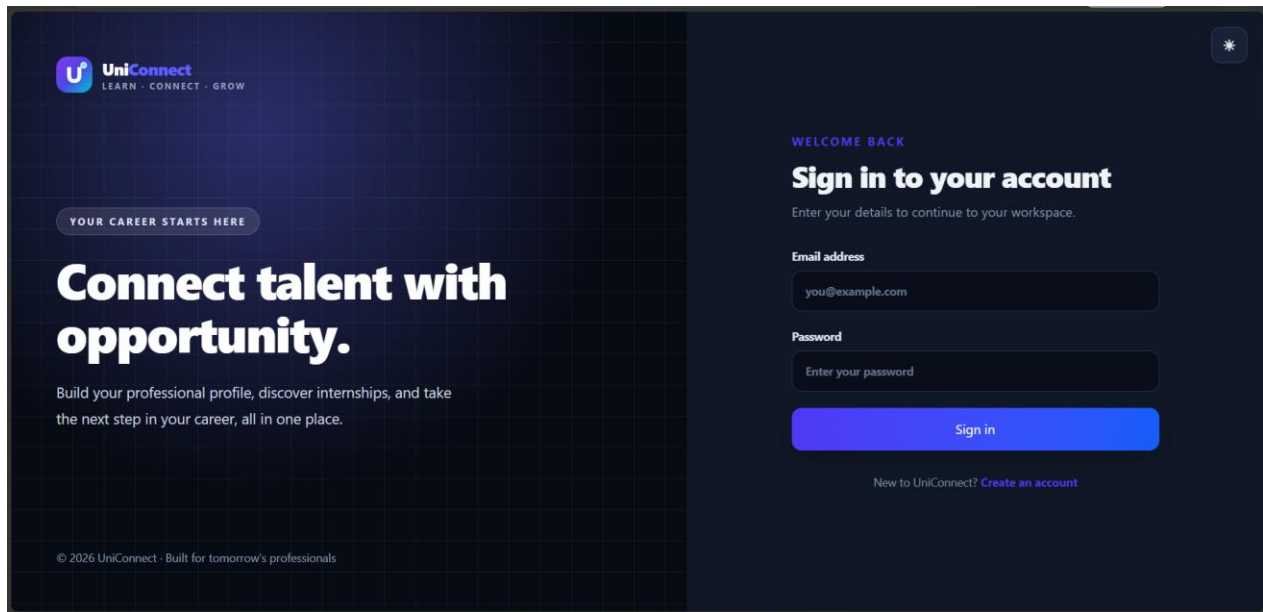


Figure 5. Sign-in screen

The opening screen introduces the product and keeps authentication focused, with direct access to account creation and theme selection.

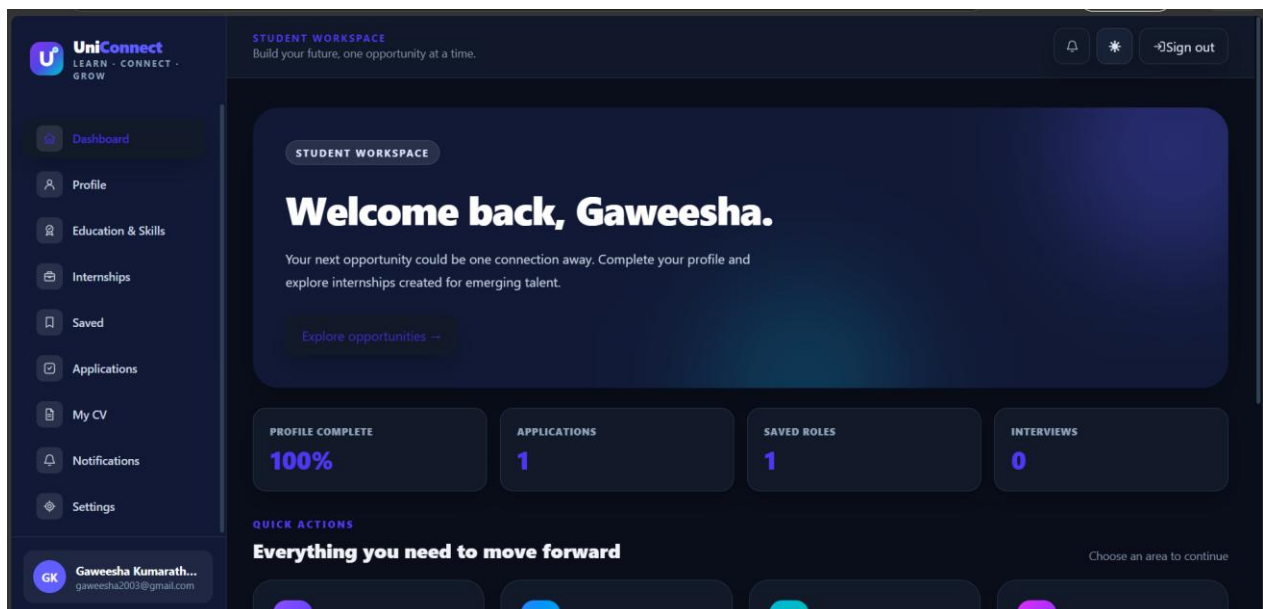


Figure 6. Student dashboard

The student workspace summarizes profile completion, applications, saved roles, and interview activity before presenting the next useful actions.

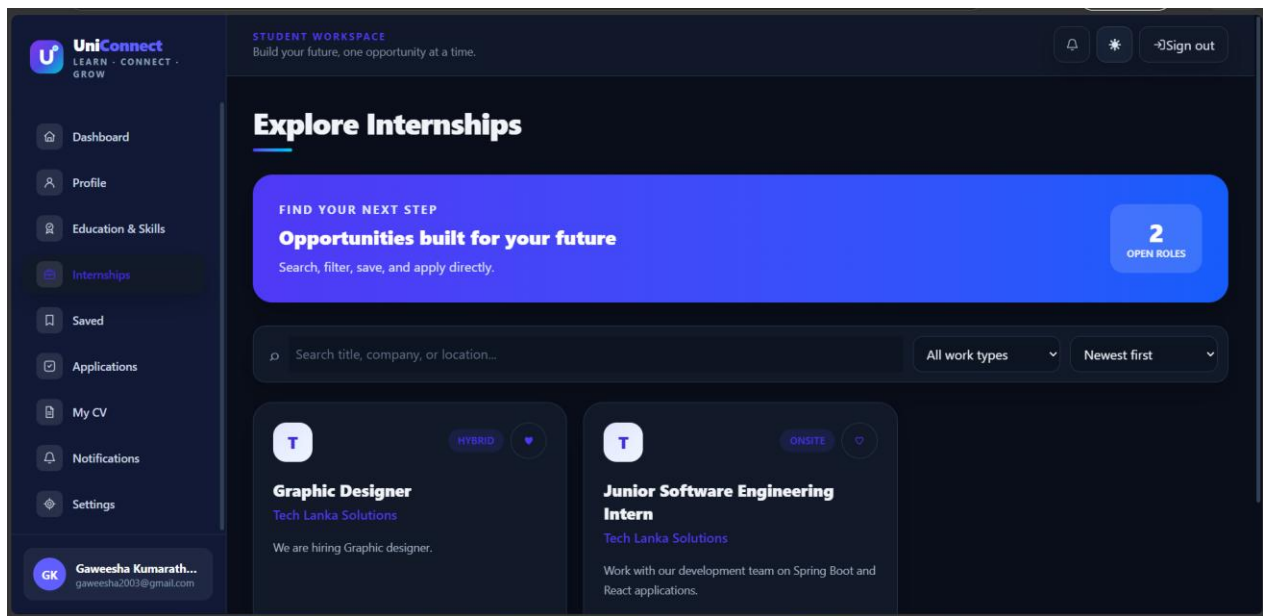


Figure 7. Internship discovery

Students can search by title, company, or location, filter by work type, sort the results, open details, and save interesting roles.

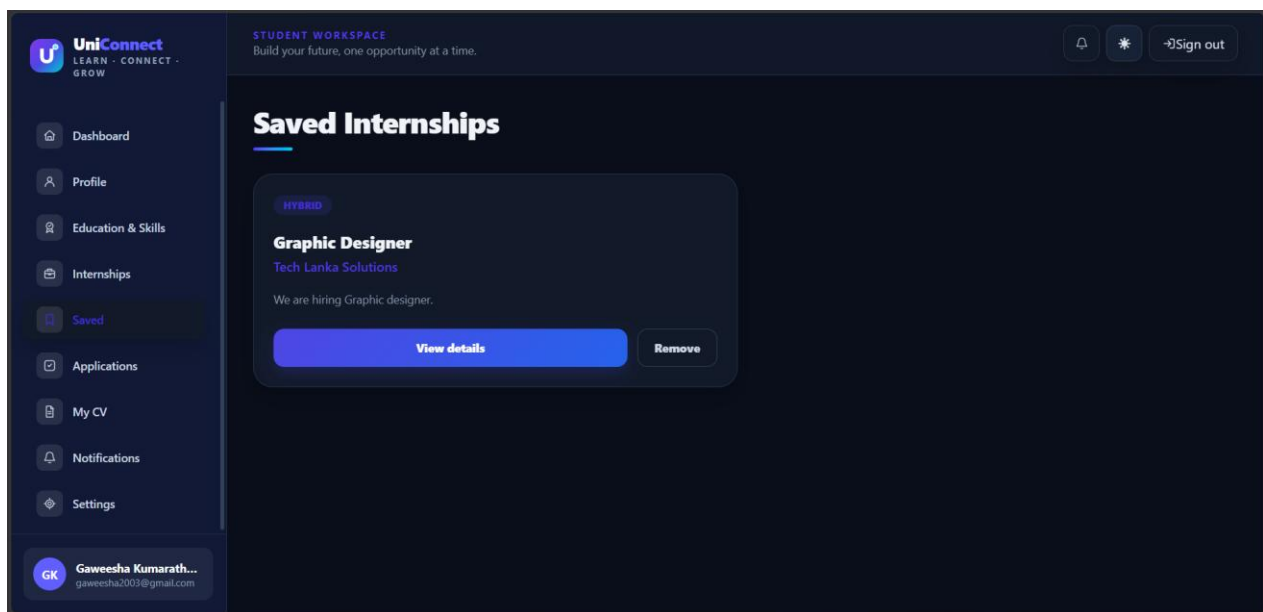


Figure 8. Saved internships

Bookmarked opportunities remain in one focused view, where the student can open the details or remove an item.

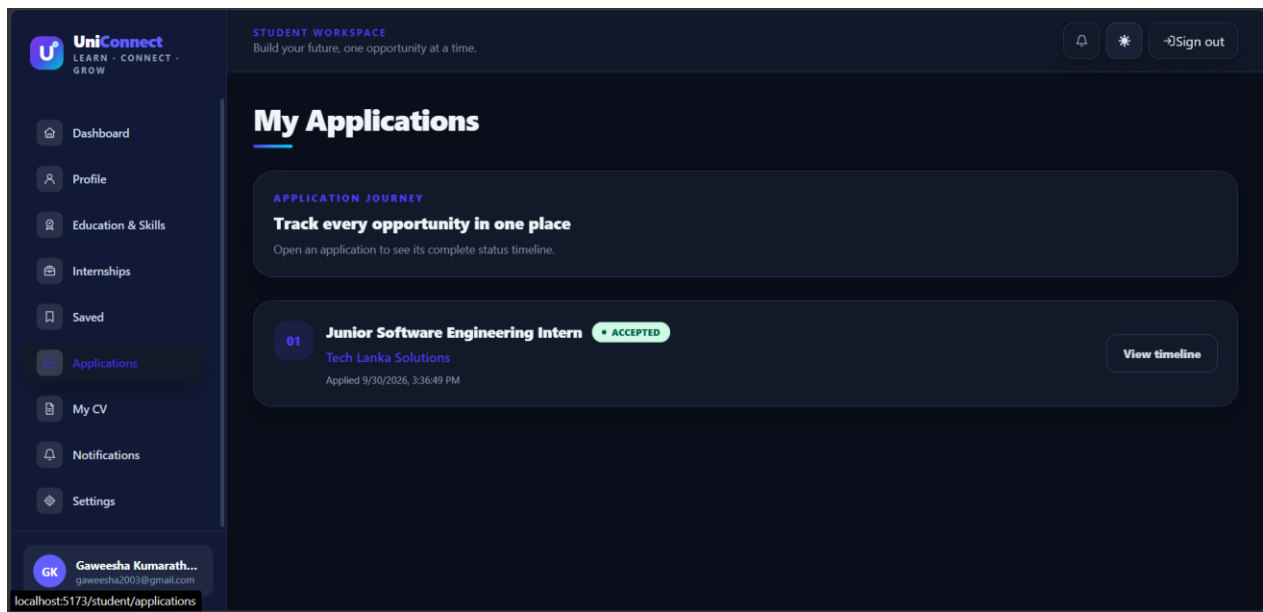


Figure 9. Student application tracking

The application list makes the current status visible and provides access to the complete decision timeline.

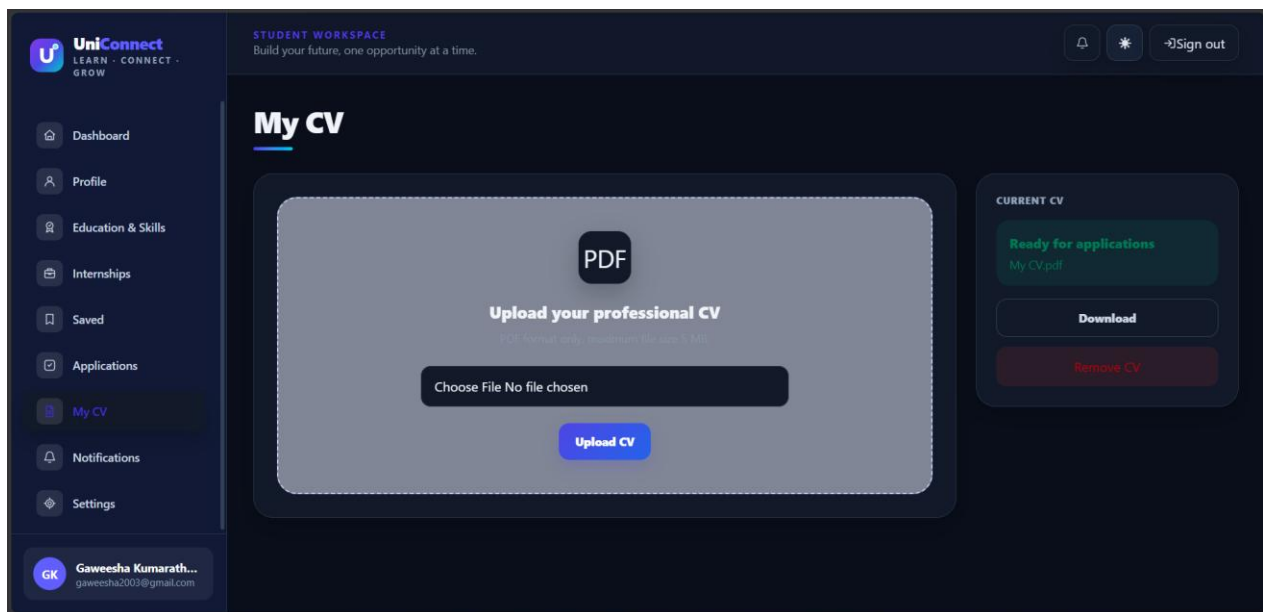


Figure 10. CV management

The CV page accepts a PDF file, shows the current document, and provides download and removal actions.

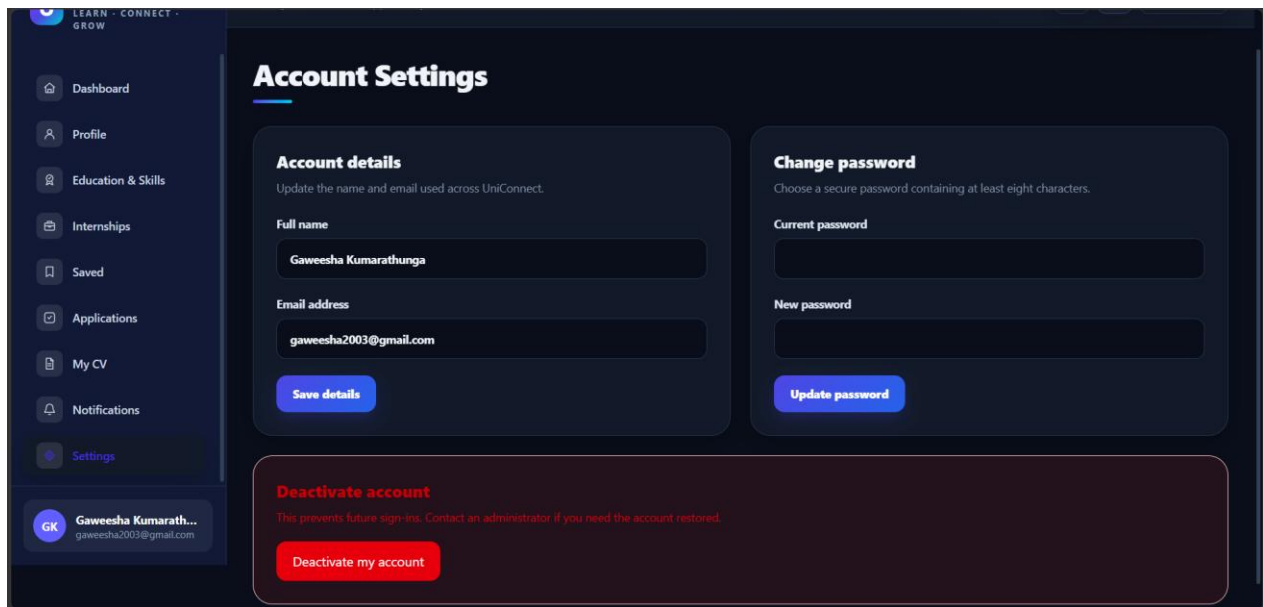


Figure 11. Account settings

A user can update shared account details, change the password, or deactivate the account from one protected page.

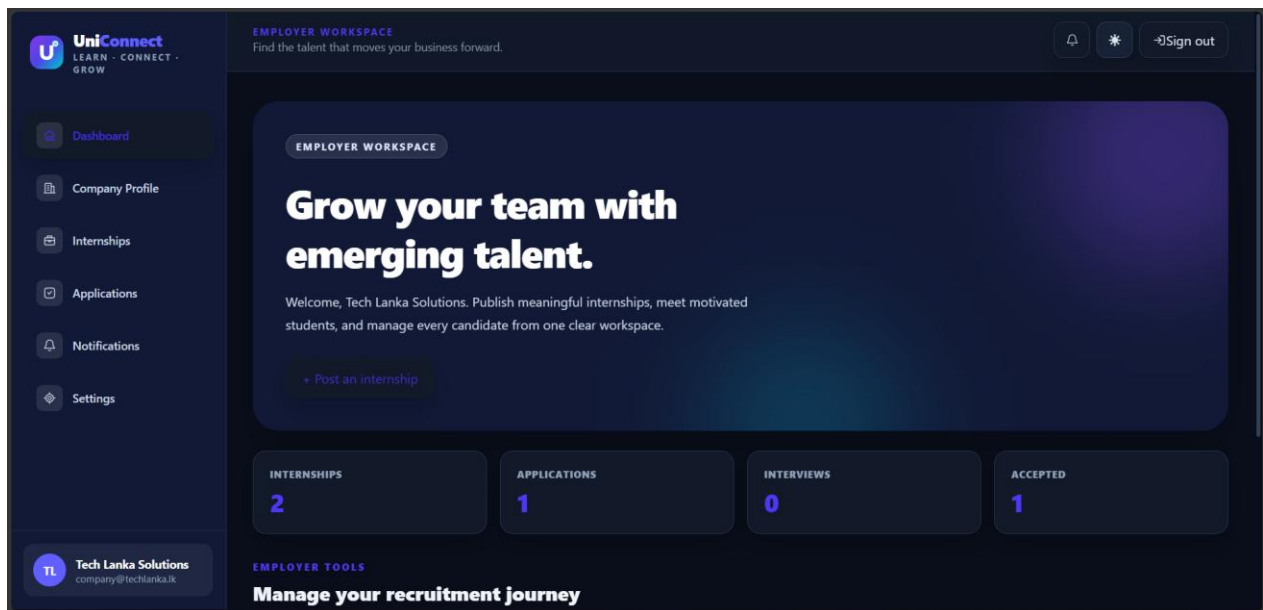


Figure 12. Employer dashboard

The employer workspace presents recruitment totals and a clear starting point for publishing a new internship.

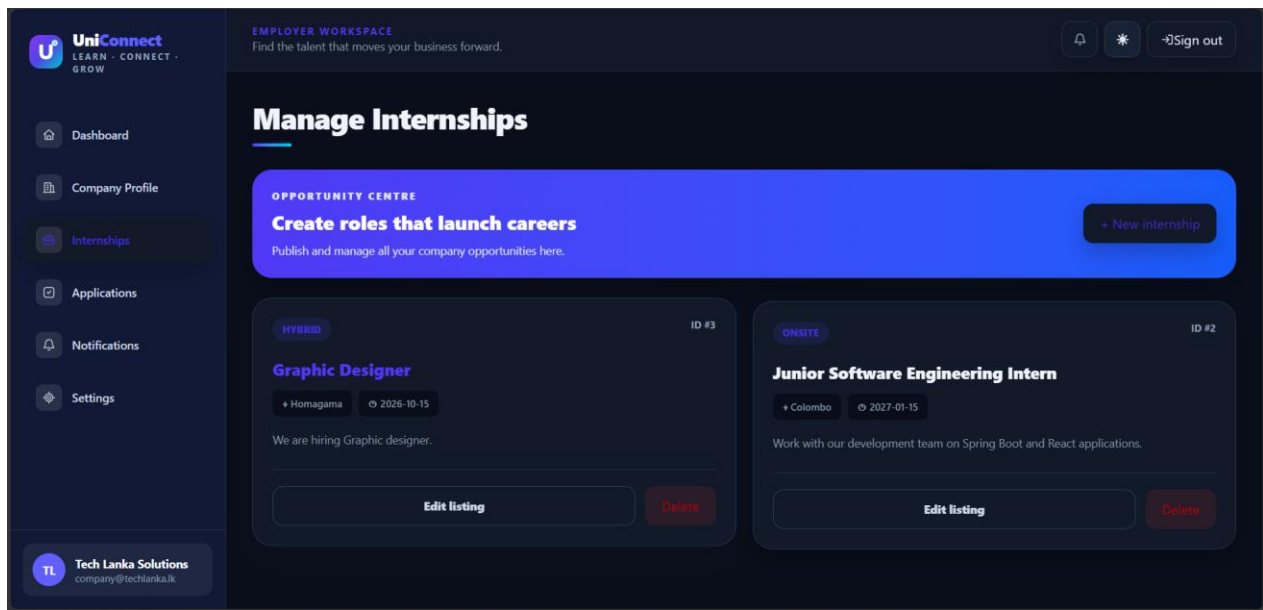


Figure 13. Company internship management

Companies can review every owned listing and use the edit, delete, or create controls without leaving the management view.

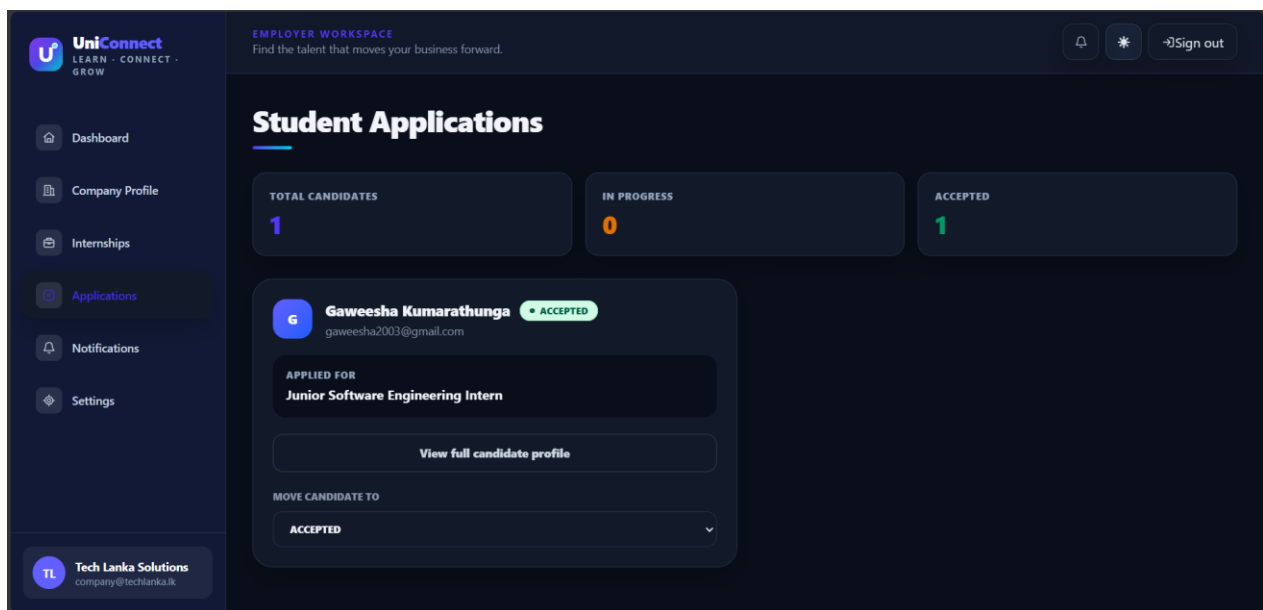


Figure 14. Candidate review

The employer can review the candidate and applied role, open the complete profile, and move the application to the next status.

11. Installation and Local Setup

11.1 Prerequisites

- Java Development Kit 24
- MySQL Community Server 8
- Node.js and npm
- Git
- IntelliJ IDEA or another Java IDE
- MySQL Workbench and Postman, optional but useful

11.2 Create the Database

```
CREATE DATABASE uniconnect
CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci;
```

11.3 Configure Windows Environment Variables

Run the following commands in PowerShell and replace the example values. Close and reopen IntelliJ after setting permanent variables so the backend process receives them.

```
[Environment]::SetEnvironmentVariable("DB_PASSWORD", "YOUR_MYSQL_PASSWORD", "User")
[Environment]::SetEnvironmentVariable("JWT_SECRET", "A_RANDOM_SECRET_WITH_AT_LEAST_48_CHARACTERS", "User")
```

11.4 Run the Backend

```
cd backend
.\mvnw.cmd spring-boot:run
```

11.5 Run the Frontend

```
cd frontend
npm install
npm run dev
```

Open <http://localhost:5173> in a browser. The frontend expects the backend at <http://localhost:8080> unless the `VITE_API_BASE_URL` setting is changed.

12. Testing and Verification

Testing focused on both the happy path and the rules that protect data. Postman was used to verify REST requests, while the browser was used to confirm complete student and employer journeys. Build and lint commands provide an additional check before committing changes.

Scenario	Expected result
Register and sign in as a student	The API returns a student JWT and the frontend opens the student dashboard.
Open a protected route without a token	The request is rejected and the user is redirected to sign in.
Create and update a student profile	The saved profile is returned for the authenticated student.
Upload an unsupported CV	The API rejects a non-PDF or a PDF larger than 5 MB.
Apply twice to the same internship	The second attempt is rejected as a duplicate.
Apply after the deadline	The application is rejected because the listing is closed.
Company updates a candidate status	The new status is saved, history is recorded, and the student receives a notification.
Company edits another company's listing	The request is rejected by the ownership rule.
Deactivate an account	Future sign-ins and protected requests for that account are blocked.
Open the interface on a phone-sized screen	Navigation changes to a mobile drawer and content stacks without horizontal overflow.

12.1 Quality Commands

```
# Frontend
cd frontend
npm run lint
npm run build

# Backend
cd backend
.\mvnw.cmd test
.\mvnw.cmd package
```

13. Limitations and Future Development

13.1 Current Limitations

- The default service addresses and CORS configuration are intended for local development.
- CV files are kept on the local backend file system rather than cloud object storage.
- Email verification, forgotten-password recovery, and refresh tokens are not implemented yet.
- Search, filters, sorting, and pagination are mainly handled in the frontend.
- Automated test coverage should be expanded before a production deployment.

13.2 Recommended Next Steps

- Deploy the frontend, API, database, and CV storage to managed production services.
- Add email verification, password recovery, refresh tokens, and optional multi-factor authentication.
- Move filtering, sorting, and pagination to server-side queries for larger datasets.
- Add skill-based recommendations and richer company verification.
- Introduce audit logs, rate limiting, monitoring, scheduled backups, and stronger automated tests.
- Run a formal accessibility review and user test with students and recruiters.

14. Conclusion

UniConnect demonstrates a complete internship workflow rather than a collection of isolated pages. A student can prepare a profile, discover an opportunity, submit an application, and follow the outcome. A company can publish that opportunity, understand the candidate, and record a decision. The supporting security, validation, database relationships, notifications, responsive interface, and administrator controls make those journeys dependable.

The current version is suitable as an academic and portfolio project and provides a strong foundation for future production work. Its clearest strength is the way the frontend, API, and database use the same domain language, which makes the experience easier to understand for users and the codebase easier to maintain for developers.

Appendix A. Quick Role Guide

Role	Start here	Typical next action
Student	Complete Profile, Education and Skills, and My CV.	Browse internships, save suitable roles, and apply.
Company	Complete Company Profile.	Publish an internship and review incoming applications.
Administrator	Review platform statistics.	Investigate users or listings only when moderation is needed.

Appendix B. Repository

Source repository: <https://github.com/Kumarathunga2003/UniConnect>

Prepared by Gaweeshha Bhagyajith Kumarathunga on 8 October 2026.